# Leitura guiada do `lib.py`

Núcleo de cálculo — lugar das raízes e discretização

PSC — Controle Linear

14 de outubro de 2026

**Resumo**

Este documento percorre, *trecho por trecho*, o arquivo `lib.py` (1189 linhas). Para cada bloco são ditos três coisas: **o que faz**, **qual sintaxe Python/SymPy usa** e **qual é a lógica matemática**. As listagens são inseridas direto do arquivo-fonte com `\lstinputlisting`, de modo que o relatório nunca diverge do código real.

## Sumário

1 Visão geral 2

2 Imports, símbolos e superfície pública 2

3 Utilitários 3

4 Validação 4

5 Especificações de desempenho → polo desejado 5

6 Montagem da planta 6

7 Passos 11 e 12 — tabela do LGR 7

8 Motor de síntese 8

9 Discretização 17

10 Extensões 22

11 Por que não há `__all__` 23

12 Mapa de chamadas 23

13 Desvios em relação ao `pid.ipynb` 23

# 1 Visão geral

O módulo tem três responsabilidades:

1. **Construir** a função de transferência em malha aberta a partir de coeficientes digitados por texto.
2. **Sintetizar** o controlador (P, PI, PD, PID ou genérico) que coloca o polo dominante no ponto desejado, impondo a condição de ângulo (passo 12) e a condição de módulo (passo 11).
3. **Discretizar** planta e controlador por Euler progressivo, Euler regressivo ou trapezoidal (Tustin).

   **Contrato com o `app.py`.** `lib.py` não importa `streamlit` e não imprime nada por conta própria: os avisos do notebook são acumulados numa lista `avisos` e devolvidos ao chamador. Isso mantém a matemática testável fora do navegador.

**Teoria.** Passo 11/12 do LGR em *SisContr.pdf* cap. 2 (PDF p. 21); especificações de desempenho em cap. 4 (PDF p. 32–37); discretização em cap. 5 (PDF p. 55–57); complemento em Ogata cap. 5 (p. 160–173) e cap. 6 (p. 258–260, p. 295–303).

# 2 Imports, símbolos e superfície pública

**1–21** (linhas 1–7, 8–14, 15–21)

```
"""
lib.py --- Núcleo de cálculo do projeto de controladores pelo método do lugar das raízes.

Calcula e valida; a exibição é feita por 'app.py'.
"""
```

```
import cmath
import math

import numpy as np
import sympy as sp
from sympy.physics.control import Series, TransferFunction
```

```
# ---------------------------------------------------------------------------
# 1. Imports e símbolos
# ---------------------------------------------------------------------------

s, k, z = sp.symbols('s k z')
```

**Sintaxe.**

- A primeira listagem é a *docstring do módulo* (aspas triplas): o texto que aparece em `help(lib)` e nos editores. Não é executada.
- `import x` / `from m import a, b` — `cmath` e `math` são da biblioteca padrão e servem para fase em ponto flutuante (rápido); `numpy` (`np`) para `sqrt`, `log`, `degrees`, `angle`; `sympy` (`sp`) para a álgebra simbólica.
- `s, k, z = sp.symbols('s k z')` — *desempacotamento de tupla*: uma chamada cria três símbolos independentes. `s` é a variável contínua, `z` a discreta, `k` o ganho do LGR.
- **Não há `__all__`.** A superfície pública é definida só pela convenção do sublinhado: nomes começam com `_` são internos, os demais são públicos. Ver §11 para a justificativa.

**Lógica.** `Series` e `TransferFunction` vêm de `sympy.physics.control`: são os objetos que permitem montar $G(s)H(s)$ e multiplicá-lo. (`pole_zero_numerical_data` também estava importado e foi removido — o módulo não extrai polos; quem faz isso é o `app.py`, que tem o import próprio.)

# 3 Utilitários

As funções `polinomio`, `angulo` e `exibir_ft` do `pid.ipynb` *não foram portadas*: uma instrumentação do app (21 execuções, 5 tipos de controlador × 7 polos) registrou **zero chamadas** às três. O que o app usa no lugar está descrito em §3.

## 2.1 `_simplificar` (linhas 105–114)

```
# ------------------------------------------------------------------------
# 2. Utilitários
# ------------------------------------------------------------------------

def _simplificar(expr):
    """Reduz a expressão a uma fração irredutível, sem alterar a sua equivalência.

    `sp.cancel`/`sp.together` apenas reorganizam e cancelam fatores comuns,
    garantindo que a FT discreta volte a ser uma razão de polinômios em `z`.
    """
    try:
        return sp.cancel(sp.together(expr))
    except (sp.PolynomialError, NotImplementedError, TypeError, ValueError):
        return sp.simplify(expr)
```

**Sintaxe.** `tuple` implícita de exceções no `except` (`(A, B, C)`) permite capturar várias exceções de uma vez; `except ... : return ...` sem `raise` transforma erro em fallback.

**Lógica.** `together` junta tudo num único denominador e `cancel` cancela fatores comuns — juntos garantem que a FT discreta volte a ser uma razão de polinômios em $z$. Não altera a equivalência, só a forma.

## 2.2 `_expressao_s` (linhas 117–131)

```
def _expressao_s(ft):
    """Normaliza uma FT contínua como uma expressão SymPy em `s`.

    Atenção: `TransferFunction.subs(...)` ignora substituições por expressões
    arbitrárias, por isso a conversão é feita com `to_expr()`.
    """
    if ft is None:
        raise ValueError("A função de transferência contínua não foi informada.")

    if hasattr(ft, "doit") and hasattr(ft, "to_expr"):
        expr = ft.doit().to_expr()
    else:
        expr = sp.sympify(ft)

    return _simplificar(expr)
```

**Sintaxe.** `hasattr(x, "doit")` sonda se o objeto é um `TransferFunction` (duck typing); `raise ValueError(...)` sinaliza entrada inválida.

**Lógica.** Converte a FT contínua em expressão SymPy em $s$. O comentário registra um detalhe do SymPy: `TransferFunction.subs(...)` *ignora* substituições por expressões arbitrárias, então é obrigatório passar por `to_expr()`.

## 2.3 `_variavel_continua` (linhas 134–155)

```
def _variavel_continua(expr):
    """Identifica o símbolo da variável contínua (por convenção, `s`) de uma expressão."""
    candidatos = expr.free_symbols - {z}

    if not candidatos:
        raise ValueError(
            "Não foi encontrada nenhuma variável contínua (s) na função de "
            "transferência informada. Expressões constantes (como o ganho puro "
            "de um controlador P) não precisam de discretização."
        )

```

```
    for simbolo in candidatos:
        if simbolo.name == "s":
            return simbolo

    if len(candidatos) == 1:
        return candidatos.pop()

    raise ValueError(
        "A função de transferência deve conter apenas a variável contínua (s). "
        f"Símbolos encontrados: {sorted(str(v) for v in candidatos)}"
    )
```

**Sintaxe.** `free_symbols` é o conjunto de símbolos de uma expressão; set - com `{z}`, `.pop()`, `sorted()` e *f-strings* com `{str(v) for v in ...}` (compreensão de conjunto).

**Lógica.** Descobre o símbolo da variável contínua. A regra é: primeiro procura o que se chama literalmente `s`; se houver exatamente um candidato ele é aceito; caso contrário é erro. Um controlador P puro (ganho constante) não tem símbolo nenhum e cai no `raise` com mensagem explicativa.

# 4 Validação

## 3.1 `_validar_periodo` (linhas 162–184)

```
# ------------------------------------------------------------------------
# 3. Validação
# ------------------------------------------------------------------------

def _validar_periodo(Ts):
    """Valida que o período de amostragem 'Ts' é real, finito e positivo (Ts > 0)."""
    try:
        periodo = sp.sympify(Ts)
    except (sp.SympifyError, TypeError, ValueError):
        raise ValueError(
            f"Ts inválido: {Ts!r}. Informe um número real positivo."
        ) from None

    if not isinstance(periodo, sp.Expr):
        raise ValueError(f"Ts inválido: {Ts!r}. Informe um número real positivo.")

    if periodo.is_real is False or periodo.is_infinite is True:
        raise ValueError(
            f"Ts inválido: {Ts!r}. O período de amostragem deve ser real e finito."
        )

    if periodo.is_positive is not True:
        raise ValueError(
            f"Ts inválido: {Ts!r}. O período de amostragem deve ser positivo (Ts > 0)."
        )

    return periodo
```

**Sintaxe.** `raise ... from None` suprime o encadeamento de exceções (*chaining*); atributos SymPy `.is_real`, `.is_infinite`, `.is_positive` — todos devolvem `None` quando indefinidos, por isso a comparação é *estrita* (`is not True`).

**Lógica.** Garante $T_s > 0$, real e finito. O teste `is not True` em vez de `not ...` trata o caso "SymPy não sabe" como falha, que é o comportamento seguro.

## 3.2 `_validar_numero` (linhas 187–204)

```
def _validar_numero(valor, nome, minimo=None, maximo=None, exclusive=False):
    """Número real dentro de uma faixa."""
    try:
        valor = float(valor)
    except (TypeError, ValueError):
        raise ValueError(f"{nome} inválido: {valor!r}. Informe um número real.")

    if valor != valor:  # NaN
        raise ValueError(f"{nome} inválido: {valor!r}.")
    if minimo is not None:
```

```
        if exclusive and valor <= minimo:
            raise ValueError(f"{nome} inválido: {valor!r}. Deve ser > {minimo}.")
        if not exclusive and valor < minimo:
            raise ValueError(f"{nome} inválido: {valor!r}. Deve ser >= {minimo}.")
    if maximo is not None and valor >= maximo:
        raise ValueError(f"{nome} inválido: {valor!r}. Deve ser < {maximo}.")

    return valor
```

**Sintaxe.** `float()` como conversor tolerante; `valor != valor` é o teste clássico de `NaN` (nan é o único valor diferente de si mesmo); parâmetros opcionais `minimo`/`maximo` com `None` como "sem limite"; `exclusive` troca `<` por `<=`.

**Lógica.** Validador genérico de faixa. Cobre $0 < M_p < 100$, $0 < \xi \leq 1$, $\tau > 0$ etc., sempre com mensagem em português.

### 3.3 `_avisar` (linhas 207–212)

```
def _avisar(avisos, mensagem):
    """Acumula o aviso em 'avisos', ou imprime se a lista for 'None'."""
    if avisos is None:
        print(mensagem)
    else:
        avisos.append(mensagem)
```

**Sintaxe.** *Dispatching* por identidade: `if avisos is None` (comparação de identidade, não de valor).

**Lógica.** Este é o *desvio D1* em relação ao `pid.ipynb`: onde havia `print`, agora há `_avisar`. Se a lista foi passada, o aviso é acumulado e o `app.py` o transforma em `st.warning`; se não, é impresso — comportamento original do notebook.

## 5 Especificações de desempenho → polo desejado

### 4.1 `pole_from_Mp` (linhas 219–238)

```
# ------------------------------------------------------------------------
# 4. Especificações de desempenho -> polo desejado
# ------------------------------------------------------------------------

def pole_from_Mp(M_p, tol, tau):
    """Calcula polo desejado a partir de sobressinal máximo e tempo de acomodação.

    SisContr.pdf, PDF p. 33, cap. 4, sec. 4.2.
    """
    Mp = _validar_numero(M_p, 'Sobressinal', 0, 100, exclusive=True)
    tau = _validar_numero(tau, 'Tempo de acomodação', 0, None, exclusive=True)

    xi = np.sqrt(np.log(Mp / 100) ** 2 / (np.pi ** 2 + np.log(Mp / 100) ** 2))
    if tol == 2:
        omega_n = 4 / (tau * xi)
    elif tol == 5:
        omega_n = 3 / (tau * xi)
    else:
        raise ValueError(f'Tolerância deve ser 2 ou 5, não {tol}')

    w_d = omega_n * np.sqrt(1 - xi ** 2)
    sigma = xi * omega_n

    return -sigma + w_d * 1j
```

**Sintaxe.** `np.sqrt`, `np.log`, `np.pi` — numpy com vetorização; `raise` dentro de `else` de um `if/elif` (o `tol` tem de ser 2 ou 5); `.2f` e `:.2f` nos avisos.

**Lógica.** Resolve o par (sobresinal, tempo de acomodação):

$$\xi = \frac{\ln(M_p/100)}{\sqrt{\pi^2 + \ln^2(M_p/100)}}, \qquad \omega_n = \frac{c}{\tau\,\xi}, \quad c = 4\ (2\%)\ \text{ou}\ 5\ (5\%)$$

e devolve $s_i = -\sigma + j\omega_d$ com $\sigma = \xi\omega_n$ e $\omega_d = \omega_n\sqrt{1-\xi^2}$. Fonte: *SisContr.pdf* cap. 4 (PDF p. 33).

### 4.2 `pole_from_freq` (linhas 241–249)

```
def pole_from_freq(w_n, xi):
    """Calcula polo desejado a partir de freq. natural e amortecimento."""
    w_n = _validar_numero(w_n, 'Frequência natural', 0, None, exclusive=True)
    xi = _validar_numero(xi, 'Amortecimento', 0, 1)

    w_d = w_n * np.sqrt(1 - xi ** 2)
    sigma = xi * w_n

    return -sigma + w_d * 1j
```

**Lógica.** Mesmo par final, mas entrando por $\omega_n$ e $\xi$ dados diretamente — é o caminho inverso e mais curto.

### 4.3 `pole_from_complex` (linhas 252–254)

```
def pole_from_complex(real, imag):
    """Monta o polo desejado a partir das coordenadas."""
    return complex(float(real), float(imag))
```

**Sintaxe.** `complex(a, b)` constrói $a + jb$ a partir de duas partes reais.

### 4.4 `grandezas_polo` (linhas 257–260)

```
def grandezas_polo(ponto_si):
    """(sigma, w_d) de `p = -sigma + j*w_d`."""
    ponto_si = complex(ponto_si)
    return float(-ponto_si.real), float(ponto_si.imag)
```

**Lógica.** Projeção do ponto desejado em $(\sigma, \omega_d)$: $\sigma = -\mathrm{Re}(s_i)$, $\omega_d = \mathrm{Im}(s_i)$. O sinal de menos é o ponto que já foi *corrigido*: o `sigma` é a distância ao eixo imaginário, sempre positiva, enquanto `real` é negativo.

## 6 Montagem da planta

### 5.1 `_polinomio_de_coeficientes` (linhas 267–269)

```
# ------------------------------------------------------------------------
# 5. Montagem da planta
# ------------------------------------------------------------------------

def _polinomio_de_coeficientes(entrada):
    """Converte '1, 8, 32, 0' num polinômio em `s` (maior grau -> menor grau)."""
    return sp.Poly.from_list(entrada.split(','), gens=s).as_expr()
```

**Sintaxe.** `entrada.split(',')` divide por vírgula; `sp.Poly.from_list(lista, gens=s)` constrói o polinômio (a lista já vem na ordem maior→menor grau) e `.as_expr()` o converte para expressão.

### 5.2 `get_ft` (linhas 272–286)

```
def get_ft(g_num_str, g_den_str, h_num_str, h_den_str):
    """Monta a função de transferência em malha aberta `G(s)H(s)`.

    Os quatro parâmetros são strings com os coeficientes separados por vírgula,
    ordenados do maior para o menor grau.
    """
    numerador = _polinomio_de_coeficientes(g_num_str)
    denominador = _polinomio_de_coeficientes(g_den_str)
    G = TransferFunction(numerador, denominador, s)

    numerador = _polinomio_de_coeficientes(h_num_str)
```

```
    denominador = _polinomio_de_coeficientes(h_den_str)
    H = TransferFunction(numerador, denominador, s)

    return Series(G, H).doit()
```

**Sintaxe.** `TransferFunction(num, den, s)` e `Series(G, H).doit()` (produto de séries, depois avaliado).

**Lógica.** Monta $G(s)H(s)$ a partir dos quatro campos de texto. O `.doit()` é o passo que de fato multiplica os polinômios — sem ele o objeto fica como expressão não avaliada.

### 5.3 `ganho_planta` (linhas 289–291)

```
def ganho_planta(ft):
    """Razão dos coeficientes líderes: o K de '1 + K G(s)H(s) = 0'."""
    return sp.Poly(ft.num, s).LC() / sp.Poly(ft.den, s).LC()
```

**Sintaxe.** `sp.Poly(...).LC()` (*leading coefficient*, coeficiente de maior grau).

**Lógica.** Razão entre os coeficientes líderes: exatamente o $K$ que multiplica a FT na equação característica $1 + K\,G(s)H(s) = 0$.

## 7 Passos 11 e 12 — tabela do LGR

### 6.1 `tabela_lgr` (linhas 298–340)

```
# ------------------------------------------------------------------------
# 6. Propriedades e tabelas do LGR (Passos 11 e 12)
# ------------------------------------------------------------------------

def tabela_lgr(ponto_si, polos, zeros):
    """Passos 11 e 12: módulo, fase e déficit de ângulo em 's_i'.

    O vetor vai do zero/polo até 's_i': 's_i - z_j' e 's_i - p_k'.
    """
    ponto_si = complex(ponto_si)

    def _linha(posicao):
        vetor = ponto_si - posicao
        return {
            "posicao": posicao,
            "modulo": float(abs(vetor)),
            "fase": float(np.degrees(np.angle(vetor))),
        }

    linhas_zeros = [_linha(zj) for zj in zeros]
    linhas_polos = [_linha(pk) for pk in polos]

    produto_zeros = 1.0
    for linha in linhas_zeros:
        produto_zeros *= linha["modulo"]

    produto_polos = 1.0
    angulo_ponto = 0.0
    for linha in linhas_polos:
        produto_polos *= linha["modulo"]
        angulo_ponto -= linha["fase"]
    for linha in linhas_zeros:
        angulo_ponto += linha["fase"]

    phi = 180 - angulo_ponto
    while phi < 0:
        phi += 360
    phi = phi % 360

    return {
        "zeros": linhas_zeros,
        "polos": linhas_polos,
        "produto_zeros": produto_zeros,
        "produto_polos": produto_polos,
        "angulo_ponto": angulo_ponto,
```

```
        "phi": phi,
    }
```

**Sintaxe.** *Função aninhada* `_linha` (definida dentro de `tabela_lgr`) evita repetir o cálculo; *list comprehensions* com `for zj in zeros`; dicionário literal como retorno; `while` + `phi += 360` para normalizar o ângulo.

**Lógica.** Para cada zero $z_j$ e polo $p_k$ mede o vetor que vai até $s_i$:

$$\|s_i - z_j\|, \qquad \angle(s_i - z_j)$$

e devolve os produtos (condição de módulo), a soma das fases (condição de ângulo) e o déficit

$$\varphi = \left(180^\circ - \textstyle\sum_j \angle(s_i - z_j) + \sum_k \angle(s_i - p_k)\right) \bmod 360^\circ.$$

O passo 11 é o módulo, o passo 12 é o ângulo — o app exibe exatamente esta tabela.

# 8 Motor de síntese

## 7.1 `angulo_deficit` (linhas 347–366)

```
# ------------------------------------------------------------------------
# 7. Motor de síntese
# ------------------------------------------------------------------------

def angulo_deficit(ponto_si, polos, zeros):
    """
    Déficit de ângulo (em graus) que os zeros do controlador precisam fornecer.

    Soma as fases vistas em 'ponto_si': cada polo entra com -phase(si - p) e
    cada zero com +phase(si - z). O controlador precisa somar '180 - soma'.
    """
    angulo_ponto = 0.0

    for polo in polos:
        angulo_ponto -= math.degrees(cmath.phase(ponto_si - polo))

    for zero in zeros:
        angulo_ponto += math.degrees(cmath.phase(ponto_si - zero))

    phi = 180 - angulo_ponto
    while phi < 0:
        phi += 360

    return phi % 360
```

**Sintaxe.** `cmath.phase` (argunto em radianos) e `math.degrees` para converter; `%` módulo em `360`.

**Lógica.**

$$\varphi = \left(180^\circ + \sum_j \angle(s_i - z_j) - \sum_k \angle(s_i - p_k)\right) \bmod 360^\circ$$

Ou seja: quanto de fase o controlador ainda precisa *somar* para fechar o laço em $s_i$. É o número que o PID precisa distribuir entre seus zeros.

## 7.2 `_forma_fechada` (linhas 369–390)

```
def _forma_fechada(phi_por_elemento, sigma, w_d, tipo, n, avisos):
    """Forma fechada da posição de N elementos com razões iguais.

    SisContr.pdf, PDF p. 34 (exemplo PD). Estável em φ = 90° (elemento na
    origem) e com erro claro quando φ → 180° (posição tende ao infinito).
    """
    if abs(phi_por_elemento - 90.0) < 1e-9:
        return sigma
    if phi_por_elemento >= 180.0 - 1e-9:
        raise ValueError(
```

```
            f"Ângulo de {phi_por_elemento:.3f}° por {tipo} não é realizável: "
            "a posição tenderia ao infinito. Escolha outro polo dominante ou "
            "use um controlador com mais elementos."
        )
    if phi_por_elemento > 90:
        return sigma - (w_d / math.tan(math.radians(180 - phi_por_elemento)))
    if phi_por_elemento <= 0:
        raise ValueError(
            f"Ângulo de {phi_por_elemento:.3f}° por {tipo} não é realizável: "
            "o elemento tende à origem."
        )
    return sigma + (w_d / math.tan(math.radians(phi_por_elemento)))
```

**Sintaxe.** `abs(x - 90.0) < 1e-9` é um teste de tolerância para comparar ponto flutuante; `math.radians` + `math.tan`; *early return*.

**Lógica.** Para $n$ elementos com razões iguais, cada um precisa contribuir $\varphi/n$, e daí sai a forma fechada

$$z = |\sigma| + \frac{\omega_d}{\tan(\varphi/n)} \quad (\varphi \le 90^\circ), \qquad z = |\sigma| - \frac{\omega_d}{\tan(180^\circ - \varphi)} \quad (\varphi > 90^\circ).$$

Os dois extremos são tratados: $\varphi = 90^\circ$ dá $z = \sigma$ (elemento na origem) e $\varphi \to 180^\circ$ faz $z \to \infty$, que vira erro explicito em vez de `nan`.

### 7.3 `posicoes_zeros` (linhas 393–480)

```
φφ→
def posicoes_zeros(phi, sigma, w_d, relacoes, avisos=None):
    """
    Posiciona N zeros reais no eixo negativo fechando o déficit 'phi' (graus).

    Cada zero em '-z' contribui 'atan2(w_d, z - sigma)' no ponto desejado e os
    zeros ficam em 'zeros[i] = relacoes[i] * z1', sendo 'relacoes[0]' a base.

    Um zero (ou todas as razões iguais) usa a forma fechada
    'z = |sigma| + w_d/tan(phi/n)'; nos demais casos resolve por bissecção, já que
    a soma das contribuições é monotônica e decrescente em z1.

    Retorna
    -------
    list de float
        Posições [z1, z2, ...] dos zeros.
    """
    relacoes = [float(razao) for razao in relacoes]
    n_zeros = len(relacoes)

    if n_zeros < 1:
        raise ValueError("O controlador precisa de pelo menos um zero.")
    if any(razao <= 0 for razao in relacoes):
        raise ValueError("As razões entre os zeros devem ser positivas (r > 0).")

    phi_graus = float(phi)
    sigma = abs(float(sigma))
    w_d = float(w_d)

    if phi_graus <= 0 or phi_graus >= 360:
        raise ValueError(
            f"Déficit de ângulo inválido: phi = {phi_graus:.2f}°. "
            "Não é possível fechar o laço com zeros reais no eixo negativo."
        )

    # Soma máxima possível: todos os zeros tendendo à origem (z -> 0+)
    angulo_maximo = n_zeros * math.degrees(math.atan2(w_d, -sigma))

    # Caso particular: um zero ou todas as razões iguais -> forma fechada
    if n_zeros == 1 or all(razao == relacoes[0] for razao in relacoes):
        posicoes = [_forma_fechada(phi_graus / n_zeros, sigma, w_d, "zero",
                                   n_zeros, avisos)] * n_zeros

        if not 0 < phi_graus < angulo_maximo:
            _avisar(
                avisos,
                f"Aviso: déficit de ângulo ({phi_graus:.2f}°) fora do intervalo "
```

```
                f"que zeros reais fecham (0° < phi < {angulo_maximo:.2f}°); "
                "o resultado segue a fórmula fechada do projeto original."
            )
        elif any(zero <= 0 for zero in posicoes):
            _avisar(avisos, f"Aviso: zero do controlador em posição não positiva ({posicoes[0]:.4f}).")

        return posicoes

    if phi_graus >= angulo_maximo:
        raise ValueError(
            f"Déficit de ângulo inviável: phi = {phi_graus:.2f}° exige mais de "
            f"{n_zeros} zero(s) (máximo de {angulo_maximo:.2f}°). "
            "Aumente o número de zeros ou use um projeto diferente."
        )

    # Bissecção: soma_angulos decresce de angulo_maximo até 0 quando z1 cresce
    def soma_angulos(z1):
        return sum(
            math.degrees(math.atan2(w_d, razao * z1 - sigma)) for razao in relacoes
        )

    limite_inferior = 1e-12
    limite_superior = max(sigma, 1.0)
    for _ in range(200):
        if soma_angulos(limite_superior) < phi_graus:
            break
        limite_superior *= 2
    else:
        raise ValueError("Não foi possível delimitar a busca do primeiro zero.")

    for _ in range(300):
        meio = 0.5 * (limite_inferior + limite_superior)
        if soma_angulos(meio) > phi_graus:
            limite_inferior = meio
        else:
            limite_superior = meio
        if limite_superior - limite_inferior < 1e-14 * max(1.0, abs(limite_superior)):
            break

    z1 = 0.5 * (limite_inferior + limite_superior)

    return [z1 * razao for razao in relacoes]
```

**Sintaxe.** `all(...)` e `any(...)` com gerador; lista compreensiva `[ expr ] * n` (repete o mesmo valor $n$ vezes); *bissecção* com `for _ in range(...)` e `break`; `range` aninhado para a busca do limite superior.

**Lógica.** Dois caminhos:

1. *Forma fechada* — um zero só ou todas as razões iguais (`all(razao == relacoes[0] ...)`). É o caso do PI e do PID padrão.

2. *Bissecção* — razões diferentes. A soma $S(z_1) = \sum_i \arctan 2(\omega_d, r_i z_1 - \sigma)$ é monótona decrescente em $z_1$, então o método da bisseção converge. O intervalo inicial $[10^{-12}, \max(\sigma, 1)]$ é *dobrado* até conter a raiz.

A linha `angulo_maximo` calcula o teto físico: com todos os zeros na origem a soma máxima é $n \arctan 2(\omega_d, -\sigma)$. Se $\varphi$ exceder isso, nenhum zero real resolve — e o código *avisa* (não quebra), reproduzindo o comportamento do notebook.

### 7.4 `coeficientes_ft` (linhas 483–501)

```
φφ→
def coeficientes_ft(expr, var=s):
    """
    Coeficientes do numerador e do denominador de uma FT (ordem decrescente).

    O formato é o mesmo aceito por '_polinomio_de_coeficientes': o primeiro
    número é o coeficiente de maior grau. Use as listas 'numerador'/'denominador'
    quando houver zeros no meio ou no fim (por exemplo o denominador s de um
    PI ou PID).
```

```
    """
    numerador, denominador = sp.fraction(sp.cancel(sp.together(expr)))
    polinomio_num = sp.Poly(numerador, var)
    polinomio_den = sp.Poly(denominador, var)

    return {
        "numerador": polinomio_num.all_coeffs(),
        "denominador": polinomio_den.all_coeffs(),
        "texto_numerador": " ".join(sp.sstr(c) for c in polinomio_num.all_coeffs()),
        "texto_denominador": " ".join(sp.sstr(c) for c in polinomio_den.all_coeffs()),
    }
```

**Sintaxe.** `sp.fraction(...)` separa numerador e denominador; `all_coeffs()` devolve os coeficientes em ordem decrescente de grau; *generator expression* dentro de `.join(...)`.

**Lógica.** Achata a FT no formato "1, 8, 32, 0" que o app mostra no *popover* de coeficientes e que `get_ft` sabe ler de volta. Mantém as listas `numerador`/`denominador` porque `get_ft` descarta zeros — justamente o caso do denominador $s$ de um PI/PID.

### 7.5 `calcular_controlador` (linhas 504–654)

```
φφ→
def calcular_controlador(ft, ponto_si, polos, zeros, sigma=None, w_d=None,
                         n_zeros=1, relacoes=None, polos_denominador=(),
                         avisos=None):
    """
    Calcula C(s) = ∏K·(s + z_i) / ∏(s - p_j) a partir do ponto dominante.

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        FT da planta em malha aberta (G·H).
    ponto_si : complex
        Polo dominante desejado.
    polos, zeros : lista
        Polos e zeros da planta (os polos do controlador já incluídos em 'polos',
        como nas células de PI/PD/PID do notebook).
    sigma, w_d : float
        Parte real e frequência natural do ponto desejado. Se 'None', são
        obtidas de 'ponto_si'.
    n_zeros : int
        Número de zeros do controlador. 'n_zeros=0' dá o P puro.
    relacoes : lista de float
        Razões entre os zeros: 'zeros[i] = relacoes[i] * zeros[0]'.
    polos_denominador : lista
        Polos (reais, não positivos) do denominador de C(s); use 0.0 para
        integradores.
    avisos : lista, opcional
        Se fornecida, os avisos são acumulados nela em vez de impressos.

    Retorna
    -------
    dict
        'zeros', 'polos', 'ganho', 'phi', 'Gc' (expressão em s), 'Gc_tf'
        (TransferFunction) e 'coeficientes'.
    """
    if sigma is None or w_d is None:
        sigma, w_d = grandezas_polo(ponto_si)

    if n_zeros < 0:
        raise ValueError("O número de zeros do controlador não pode ser negativo.")

    relacoes = [float(r) for r in (relacoes if relacoes is not None else [1.0] * n_zeros)]
    if len(relacoes) != n_zeros:
        raise ValueError(
            f"Esperadas {n_zeros} razões entre os zeros, recebidas {len(relacoes)}."
        )
    if any(r <= 0 for r in relacoes):
        raise ValueError("As razões entre os zeros devem ser positivas (r > 0).")

    polos_denominador = [complex(polo) for polo in polos_denominador]
    for polo in polos_denominador:
        if abs(polo.imag) > 1e-9:
```

```
φφ→∏∏
            raise ValueError(
                "Os polos do controlador devem ser reais: informe um polo e o "
                "seu conjugado, se quiser um par complexo."
            )
    polos_denominador = [polo.real for polo in polos_denominador]

    # --- Caso especial: controlador proporcional puro (n_zeros = 0) ---------
    if n_zeros == 0:
        numerador = sp.Integer(1)
        denominador = sp.Integer(1)
        for polo in polos_denominador:
            denominador *= (s - polo)

        magnitude = abs(
            complex(Series(ft, TransferFunction(numerador, denominador, s)).doit()
                    .eval_frequency(ponto_si))
        )
        if magnitude == 0:
            raise ValueError("Magnitude nula em si: o ganho K não pode ser calculado.")
        ganho = 1 / magnitude

        _avisar(
            avisos,
            "Aviso: controlador P não possui zero nem polo, portanto **não altera "
            "a fase**. Apenas a condição de módulo é satisfeita; confirme se p_i já "
            "pertence ao LGR da planta pura.",
        )

        return {
            "zeros": [],
            "polos": polos_denominador,
            "ganho": ganho,
            "phi": angulo_deficit(ponto_si, polos, zeros),
            "Gc": sp.together(ganho * numerador / denominador),
            "Gc_tf": TransferFunction(ganho * numerador, denominador, s),
            "coeficientes": coeficientes_ft(ganho * numerador / denominador),
        }

    if n_zeros < 1:
        raise ValueError("O controlador precisa de pelo menos um zero (n_zeros >= 1).")

    # 1) ângulo que os zeros precisam fornecer
    phi = angulo_deficit(ponto_si, polos, zeros)

    # 2) posições dos zeros
    posicoes = posicoes_zeros(phi, sigma, w_d, relacoes, avisos=avisos)

    # 3) ganho pela condição de magnitude: |C(si)·ft(si)| = 1
    if all(zero == posicoes[0] for zero in posicoes):
        numerador = (s + posicoes[0]) ** len(posicoes)
    else:
```

```
φφ→∏∏
        numerador = sp.Integer(1)
        for zero in posicoes:
            numerador *= (s + zero)

    denominador = sp.Integer(1)
    for polo in polos_denominador:
        denominador *= (s - polo)

    Gc_unitario = TransferFunction(numerador, denominador, s)
    magnitude = abs(complex(Series(ft, Gc_unitario).doit().eval_frequency(ponto_si)))
    if magnitude == 0:
        raise ValueError("Magnitude nula em si: o ganho K não pode ser calculado.")
    ganho = 1 / magnitude

    resultado = {
        "zeros": posicoes,
        "polos": polos_denominador,
        "ganho": ganho,
        "phi": phi,
        "Gc": sp.together(ganho * numerador / denominador),
```

```
        "Gc_tf": TransferFunction(ganho * numerador, denominador, s),
        "coeficientes": coeficientes_ft(ganho * numerador / denominador),
    }

    # reprodução fiel do print original (ângulo nominal de cada zero)
    angulo_primeiro_zero = math.degrees(math.atan2(w_d, posicoes[0] - abs(sigma)))
    if all(razao == relacoes[0] for razao in relacoes):
        _avisar(avisos, f"Angulo = {phi / len(posicoes):.2f}°")
        _avisar(avisos, f"Zero do controlador: {-posicoes[0]:.2f} + 0*j")
    else:
        _avisar(avisos, f"Angulo = {phi:.2f}° (repartido entre {len(posicoes)} zeros)")
        _avisar(
            avisos,
            f"Zero do controlador: {-posicoes[0]:.2f} + 0*j "
            f"(razão {relacoes[0]:.3f}, ângulo {angulo_primeiro_zero:.2f}°)"
        )

    for indice, (zero, razao) in enumerate(zip(posicoes[1:], relacoes[1:]), start=2):
        contribuicao = math.degrees(math.atan2(w_d, zero - abs(sigma)))
        _avisar(
            avisos,
            f"Zero {indice} do controlador: {-zero:.2f} + 0*j "
            f"(razão {razao:.3f}, ângulo {contribuicao:.2f}°)"
        )
    if polos_denominador:
        _avisar(avisos, f"Polos do controlador: {[round(p, 4) for p in polos_denominador]}")
    _avisar(avisos, f"Kc = {ganho}\n")

    return resultado
```

**Sintaxe.**

- *Keyword-only* não é usado, mas `n_zeros=1` etc. são passados por nome — o que torna a chamada legível mesmo com 10 parâmetros.
- `abs(polo.imag) > 1e-9` — teste de parte imaginária desprezível.
- `complex(...)` e `Series(...).eval_frequency(ponto_si)` — avaliação numérica da FT no ponto $s_i$.
- `all(zero == posicoes[0] for zero in posicoes)` — detecta zeros coincidentes para montar $(s + z_1)^n$ em vez do produto repetido.
- `zip(posicoes[1:], relacoes[1:]), start=2` — enumeração a partir do segundo elemento.

**Lógica.** É o coração do projeto. Calcula

$$C(s) = \frac{K \prod_i (s + z_i)}{\prod_j (s - p_j)}$$

em quatro passos:

1. **Ângulo**: $\varphi =$ `angulo_deficit`$(\cdot)$.
2. **Zeros**: posições via `posicoes_zeros`.
3. **Módulo**: monta $C_{\text{unit}}(s)$ com $K = 1$, avalia $|C_{\text{unit}}(s_i)G(s_i)H(s_i)|$ e toma o inverso — é a *condição de magnitude* do passo 11.
4. **Retorno**: dicionário com `zeros`, `polos`, `ganho`, `phi`, `Gc` (expressão em $s$), `Gc_tf` (`TransferFunction`) e `coeficientes`.

O **ramo** $n_zeros = 0$ (controlador P puro) é um desvio em relação ao notebook: sem zero e sem polo, $C(s) = K$, a fase não muda, e o código avisa exatamente disso.

### 7.6 `posicoes_polos` (linhas 663–744)

```
φφ→∏∏
def posicoes_polos(phi, sigma, w_d, relacoes, avisos=None):
    """Calcula a posição de N polos reais no eixo negativo para fornecer o ângulo 'phi' (graus).

    Cada polo em '-p' subtrai a fase 'atan2(w_d, p - sigma)' no ponto desejado e
    os polos ficam posicionados em 'polos[i] = relacoes[i] * p1'.
    """
    relacoes = [float(razao) for razao in relacoes]
    n_polos = len(relacoes)

    if n_polos < 1:
        raise ValueError(
            "O controlador precisa de pelo menos um polo a calcular."
        )
    if any(razao <= 0 for razao in relacoes):
        raise ValueError("As razões entre os polos devem ser positivas (r > 0).")

    phi_graus = float(phi)
    sigma = abs(float(sigma))
    w_d = float(w_d)

    if phi_graus <= 0 or phi_graus >= 360:
        raise ValueError(
            f"Ângulo de polo inválido: phi = {phi_graus:.2f}°. "
            "Não é possível fechar o laço com polos reais no eixo negativo."
        )

    angulo_maximo = n_polos * math.degrees(math.atan2(w_d, -sigma))

    # Caso particular: um polo ou todas as razões iguais -> forma fechada
    if n_polos == 1 or all(razao == relacoes[0] for razao in relacoes):
        posicoes = [_forma_fechada(phi_graus / n_polos, sigma, w_d, "polo",
                                   n_polos, avisos)] * n_polos

        if not 0 < phi_graus < angulo_maximo:
            _avisar(
                avisos,
                f"Aviso: ângulo de polo ({phi_graus:.2f}°) fora do intervalo "
                f"que polos reais fecham (0° < phi < {angulo_maximo:.2f}°)."
            )
        elif any(polo <= 0 for polo in posicoes):
            _avisar(
                avisos,
                "Aviso: polo do controlador em posição não positiva"
                f" ({posicoes[0]:.4f})."
            )

        return posicoes

    if phi_graus >= angulo_maximo:
        raise ValueError(
            f"Ângulo de polo inviável: phi = {phi_graus:.2f}° exige mais de "
            f"{n_polos} polo(s) (máximo de {angulo_maximo:.2f}°)."
        )

    def soma_angulos(p1):
        return sum(
            math.degrees(math.atan2(w_d, razao * p1 - sigma))
            for razao in relacoes
        )

    limite_inferior = 1e-12
    limite_superior = max(sigma, 1.0)
    for _ in range(200):
        if soma_angulos(limite_superior) < phi_graus:
            break
        limite_superior *= 2
    else:
        raise ValueError("Não foi possível delimitar a busca do primeiro polo.")

    for _ in range(300):
        meio = 0.5 * (limite_inferior + limite_superior)
        if soma_angulos(meio) > phi_graus:
```

```
            limite_inferior = meio
        else:
            limite_superior = meio
        if limite_superior - limite_inferior < 1e-14 * max(
            1.0, abs(limite_superior)
        ):
            break

    p1 = 0.5 * (limite_inferior + limite_superior)
    return [p1 * razao for razao in relacoes]
```

**Lógica.** Espelho exato de `posicoes_zeros`, mas o polo *subtrai* fase em vez de somar — daí a mesma estrutura com sinal trocado na expressão da soma. Serve ao controlador genérico, em que a incógnita é a posição do polo, não do zero.

### 7.7 `calcular_controlador_gen` (linhas 747–894)

```
φφ→∏∏
def calcular_controlador_gen(
    ft,
    ponto_si,
    polos,
    zeros,
    sigma=None,
    w_d=None,
    zeros_controlador=(),
    n_polos=1,
    relacoes_polos=None,
    polos_fixos=(),
    avisos=None,
):
    """Calcula C(s) = ∏K·(s + z_i) / ∏[(s + p_j)∏·(s - p_f)] a partir da condição de fase.

    Calcula a posição do(s) polo(s) do controlador necessários para alcançar o
    ponto dominante desejado 'ponto_si'.
    """
    if sigma is None or w_d is None:
        sigma, w_d = grandezas_polo(ponto_si)

    if n_polos < 1:
        raise ValueError(
            "O controlador precisa de pelo menos um polo a calcular (n_polos >= 1)."
        )

    relacoes_polos = [
        float(r)
        for r in (
            relacoes_polos if relacoes_polos is not None else [1.0] * n_polos
        )
    ]
    if len(relacoes_polos) != n_polos:
        raise ValueError(
            f"Esperadas {n_polos} razões entre os polos, recebidas {len(relacoes_polos)}."
        )
    if any(r <= 0 for r in relacoes_polos):
        raise ValueError("As razões entre os polos devem ser positivas (r > 0).")
```

```
φφ→∏∏∏∏∏

    zeros_controlador = [float(z_c) for z_c in zeros_controlador]
    polos_fixos = [complex(p) for p in polos_fixos]
    for polo in polos_fixos:
        if abs(polo.imag) > 1e-9:
            raise ValueError(
                "Os polos fixos do controlador devem ser reais."
            )
    polos_fixos = [p.real for p in polos_fixos]

    # 1) Soma da fase da planta, zeros do controlador e polos fixos
    angulo_existente = 0.0

    for polo in polos:
        angulo_existente -= math.degrees(cmath.phase(ponto_si - polo))
```

```
    for zero in zeros:
        angulo_existente += math.degrees(cmath.phase(ponto_si - zero))

    for z_c in zeros_controlador:
        angulo_existente += math.degrees(cmath.phase(ponto_si + z_c))

    for p_f in polos_fixos:
        angulo_existente -= math.degrees(cmath.phase(ponto_si - p_f))

    # O(s) polo(s) calculados do controlador precisam subtrair o ângulo 'phi_polos'
    # tal que: angulo_existente - phi_polos = 180° (mod 360°)
    phi_polos = (angulo_existente - 180) % 360
    if phi_polos == 0:
        phi_polos = 360.0

    # 2) Posições dos polos calculados (no eixo real)
    posicoes_p = posicoes_polos(phi_polos, sigma, w_d, relacoes_polos, avisos=avisos)

    # 3) Construção das expressões simbólicas de C(s)
    numerador = sp.Integer(1)
    for z_c in zeros_controlador:
```

```
φφ→ΠΠΠΠΠ
        numerador *= s + z_c

    denominador = sp.Integer(1)
    for p_calc in posicoes_p:
        denominador *= s + p_calc
    for p_f in polos_fixos:
        denominador *= s - p_f

    # 4) Ganho Kc pela condição de magnitude: |C(si)·ft(si)| = 1
    Gc_unitario = TransferFunction(numerador, denominador, s)
    magnitude = abs(
        complex(Series(ft, Gc_unitario).doit().eval_frequency(ponto_si))
    )
    if magnitude == 0:
        raise ValueError(
            "Magnitude nula em si: o ganho K não pode ser calculated."
        )
    ganho = 1 / magnitude

    polos_totais_s = [-p for p in posicoes_p] + polos_fixos

    resultado = {
        "zeros": zeros_controlador,
        "polos_calculados": posicoes_p,
        "polos": polos_totais_s,
        "ganho": ganho,
        "phi": phi_polos,
        "Gc": sp.together(ganho * numerador / denominador),
        "Gc_tf": TransferFunction(ganho * numerador, denominador, s),
        "coeficientes": coeficientes_ft(ganho * numerador / denominador),
    }

    # Exibição dos resultados
    angulo_primeiro_polo = math.degrees(
        math.atan2(w_d, posicoes_p[0] - abs(sigma))
    )
    if all(razao == relacoes_polos[0] for razao in relacoes_polos):
```

```
φφ→ΠΠΠΠΠ
        _avisar(avisos, f"Ângulo dos polos = {phi_polos / len(posicoes_p):.2f}° cada")
        _avisar(avisos, f"Polo calculado do controlador: {-posicoes_p[0]:.2f} + 0*j")
    else:
        _avisar(
            avisos,
            f"Ângulo dos polos = {phi_polos:.2f}° (repartido entre {len(posicoes_p)} polos)"
        )
        _avisar(
            avisos,
            f"Polo 1 calculado: {-posicoes_p[0]:.2f} + 0*j "
            f"(razão {relacoes_polos[0]:.3f}, ângulo {angulo_primeiro_polo:.2f}°)"
        )
```

```
    for indice, (p_val, razao) in enumerate(
        zip(posicoes_p[1:], relacoes_polos[1:]), start=2
    ):
        contribuicao = math.degrees(math.atan2(w_d, p_val - abs(sigma)))
        _avisar(
            avisos,
            f"Polo {indice} calculado: {-p_val:.2f} + 0*j "
            f"(razão {razao:.3f}, ângulo {contribuicao:.2f}°)"
        )

    if zeros_controlador:
        _avisar(
            avisos,
            f"Zeros do controlador: {[-round(z, 4) for z in zeros_controlador]}"
        )
    if polos_fixos:
        _avisar(
            avisos,
            f"Polos fixos do controlador: {[round(p, 4) for p in polos_fixos]}"
        )
    _avisar(avisos, f"Kc = {ganho}\n")

    return resultado
```

**Sintaxe.** *Lambda* implícito nas comparações ternárias; `% 360` para o módulo; `list` de `complex` convertida com `[p.real for p in ...]`.

**Lógica.** Calcula $C(s) = \dfrac{K\prod(s+z_i)}{\prod(s+p_j)\prod(s-p_f)}$ para o controlador genérico. A diferença para o caso anterior é *qual* elemento é a incógnita:

$$\varphi_p = \big(\text{fase existente} - 180^\circ\big) \bmod 360^\circ$$

onde a fase existente já inclui planta, zeros do controlador e polos fixos. O teste `if phi_polos == 0: phi_polos = 360.0` evita a divisão por zero na forma fechada (o limite $\varphi \to 360^\circ$ é o polo na origem).

### 7.8 `calcular_zc` (linhas 897–913)

```
φφ→ΠΠΠΠΠ
def calcular_zc(ft, polo_1, polos, zeros, ganho, w_d, sigma, pid, pd, pi,
                relacao_zeros=1.0):
    '''
    Calcula a posição do zero do controlador e o ganho Kc (PI/PD/PID).

    Delega para 'calcular_controlador'. 'relacao_zeros' só vale no PID.
    '''
    relacoes = [1.0, float(relacao_zeros)] if pid else [1.0]
    polos_denominador = [0.0] if (pid or pi) else []

    resultado = calcular_controlador(
        ft, polo_1, polos, zeros, sigma, w_d,
        n_zeros=len(relacoes), relacoes=relacoes,
        polos_denominador=polos_denominador,
    )

    return resultado["zeros"][0], resultado["ganho"]
```

**Sintaxe.** Tupla condicional `[0.0] if (pid or pi) else []` — escolhe se o controlador tem integrador; *destructuring* do retorno `resultado["zeros"][0], resultado["ganho"]`.

**Lógica.** É a assinatura preservada do `pid.ipynb`, usada pelas células de PI e PD. Recebe as *bandeiras* `pid/pd/pi` e devolve apenas $(z_c, K_c)$. O `relacao_zeros` só entra no PID.

## 9 Discretização

### 8.1 `_discretizar_com` (linhas 920–936)

```
φφ→ΠΠΠΠΠ
# ---------------------------------------------------------------------------
# 8. Discretização (cap. 5 do SisContr)
# ---------------------------------------------------------------------------

def _discretizar_com(ft, Ts, regra):
    """Aplica a substituição 's = regra(z, Ts)' a uma função de transferência contínua.

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        Função de transferência contínua (planta ou controlador).
    Ts : número ou símbolo positivo
        Período de amostragem.
    regra : callable
        Regra de substituição que recebe (z, Ts) e devolve a expressão para 's'.
    """
    continua = _expressao_s(ft)
    periodo = _validar_periodo(Ts)
    var = _variavel_continua(continua)

    return _simplificar(continua.subs(var, regra(z, periodo)))
```

**Sintaxe.** *Callable* passada como argumento (`regra(z, periodo)`); `.subs(var, regra(...))` faz a substituição simbólica.

**Lógica.** Núcleo único: aplica $s \leftarrow g(z, T_s)$ na FT contínua e reduz o resultado. As três funções seguintes são só a *lambda* que define $g$.

### 8.2 `discretizar_euler_progressivo` (linhas 939–961)

```
φφ→ΠΠΠΠΠ
def discretizar_euler_progressivo(ft, Ts):
    """Discretiza uma FT contínua pelo Euler progressivo (Forward Euler).

    Substituição:  s ≈ (z - 1) / Ts

    É a aproximação mais simples: 'x[k+1] ≈ x[k] + Ts·x'(t_k)'. O erro é da ordem
    de Ts e a região de estabilidade é o disco |z - 1| < 1, que não contém todo o
    plano à esquerda do eixo imaginário (pode instabilizar sistemas contínuos
    estáveis).

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        Função de transferência contínua (por exemplo 'ft' ou 'Gc').
    Ts : número real positivo
        Período de amostragem.

    Retorna
    -------
    Expr
        Função de transferência discreta em função de 'z'.
    """
    return _discretizar_com(ft, Ts, lambda z_, Ts_: (z_ - 1) / Ts_)
```

**Lógica.** Forward Euler:

$$s \approx \frac{z-1}{T_s}.$$

Erro de ordem $T_s$; região de estabilidade é o disco $|z-1| < 1$, que *não* cobre todo o semiplano esquerdo — pode instabilizar um sistema contínuo estável.

### 8.3 `discretizar_euler_regressivo` (linhas 964–985)

```
φφ→ΠΠΠΠΠ≈≈
def discretizar_euler_regressivo(ft, Ts):
    """Discretiza uma FT contínua pelo Euler regressivo (Backward Euler).

    Substituição:  s ≈ (z - 1) / (z·Ts)

```

```
    Usa a derivada no instante atual (‘x’(t_{k+1})‘), introduzindo um atraso de
    uma amostra. O erro também é da ordem de Ts, mas o método é A-estável:
    a região de estabilidade é o disco |z + 1| < 1.

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        Função de transferência contínua (por exemplo ‘ft‘ ou ‘Gc‘).
    Ts : número real positivo
        Período de amostragem.

    Retorna
    -------
    Expr
        Função de transferência discreta em função de ‘z‘.
    """
    return _discretizar_com(ft, Ts, lambda z_, Ts_: (z_ - 1) / (z_ * Ts_))
```

**Lógica.** Backward Euler:

$$s \approx \frac{z-1}{z\,T_s}.$$

Usa a derivada no instante atual, introduzindo uma amostra de atraso. Mesmo erro de ordem $T_s$, mas é *A-estável*: região $|z+1|<1$.

### 8.4 `discretizar_trapezoidal` (linhas 988–1009)

```
φφ→∏∏∏∏∏≈≈≈
def discretizar_trapezoidal(ft, Ts):
    """Discretiza uma FT contínua pelo método trapezoidal (Tustin).

    Substituição:  s ≈ (2/Ts)·(z - 1)/(z + 1)

    Aproxima a integral pela regra do trapézio, possuindo erro da ordem de Ts²
    (o dobro dos métodos de Euler) e preservando a estabilidade para Ts pequenos.
    É o método mais utilizado em controle digital.

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        Função de transferência contínua (por exemplo ‘ft‘ ou ‘Gc‘).
    Ts : número real positivo
        Período de amostragem.

    Retorna
    -------
    Expr
        Função de transferência discreta em função de ‘z‘.
    """
    return _discretizar_com(ft, Ts, lambda z_, Ts_: (2 / Ts_) * (z_ - 1) / (z_ + 1))
```

**Lógica.** Tustin:

$$s \approx \frac{2}{T_s}\,\frac{z-1}{z+1}.$$

Erro de ordem $T_s^2$ (o dobro dos Eulers) e é o método padrão em controle digital. O *warping* de frequência fica para fora do escopo do projeto.

### 8.5 Tabelas de métodos (linhas 1012–1034)

```
φφ→∏∏∏∏∏≈≈≈≈
# Cada método é implementado uma única vez; a seleção apenas encaminha a chamada.
METODOS_DISCRETIZACAO = {
    "euler": discretizar_euler_progressivo,
    "backward": discretizar_euler_regressivo,
    "trapezoidal": discretizar_trapezoidal,
}

DESCRICAO_METODOS = {
    "euler": "Euler progressivo (Forward Euler): s = (z - 1)/Ts",
    "backward": "Euler regressivo (Backward Euler): s = (z - 1)/(z*Ts)",
```

```
    "trapezoidal": "Trapezoidal (Tustin): s = (2/Ts)*(z - 1)/(z + 1)",
}

# Nomes alternativos aceitos (também em português)
ALIASES_METODOS = {
    "forward": "euler",
    "forward_euler": "euler",
    "progressivo": "euler",
    "backward_euler": "backward",
    "regressivo": "backward",
    "tustin": "trapezoidal",
    "trapz": "trapezoidal",
}
```

**Sintaxe.** Dicionários literal (nome → função) e de despacho (*alias* → nome canônico).

**Lógica.** Cada método é implementado *uma única vez*; a seleção só encaminha. `ALIASES_METODOS` aceita `forward_euler`, `progressivo`, `tustin`, `regressivo` etc., para o usuário não precisar decorar a chave exata.

### 8.6 `tornar_monico` (linhas 1037–1046)

```
φφ→ΠΠΠΠΠ≈≈≈≈
def tornar_monico(expressao, variavel):
    """Divide numerador e denominador pelo coeficiente líder do denominador."""
    num, den = sp.fraction(sp.cancel(expressao))

    coef_lider = sp.LC(sp.Poly(den, variavel))

    num = sp.expand(num / coef_lider)
    den = sp.expand(den / coef_lider)

    return num / den
```

**Sintaxe.** `sp.LC` e `sp.expand`.

**Lógica.** Divide numerador e denominador pelo coeficiente líder do denominador, deixando a FT na forma mônica $z^n + \ldots$.

### 8.7 `discretizar` (linhas 1049–1093)

```
φφ→ΠΠΠΠΠ≈≈≈≈
def discretizar(ft, Ts, metodo="trapezoidal"):
    """Discretiza 'ft' utilizando o método escolhido.

    Valores aceitos para 'metodo': 'euler', 'backward' e 'trapezoidal'
    (sem diferenciar maiúsculas/minúsculas; há apelidos em ALIASES_METODOS).

    Parâmetros
    ----------
    ft : TransferFunction ou Expr
        Função de transferência contínua.
    Ts : número real positivo
        Período de amostragem.
    metodo : str
        Método de discretização.

    Retorna
    -------
    Expr
        Função de transferência discreta em função de 'z'.

    Exemplos
    --------
    >>> discretizar(1/(s + 1), sp.Rational(1, 10), 'trapezoidal')
    (z + 1)/(21*z - 19)
    """
    if not isinstance(metodo, str):
        raise ValueError(
            f"Método de discretização inválido: {metodo!r}. "
            "Use 'euler', 'backward' ou 'trapezoidal'."
        )
```

```
32
33      nome = metodo.strip().lower()
34      nome = ALIASES_METODOS.get(nome, nome)
35
36      if nome not in METODOS_DISCRETIZACAO:
37          raise ValueError(
38              f"Método de discretização desconhecido: {metodo!r}. "
39              "Opções válidas: 'euler', 'backward' ou 'trapezoidal'."
40          )
41
42      resultado = METODOS_DISCRETIZACAO[nome](ft, Ts)
43      resultado_monico = tornar_monico(resultado, z)
44
45      # Deixar a função de transferência mônica
46      return resultado_monico
```

**Sintaxe.** `isinstance(metodo, str)`; `metodo.strip().lower()` normaliza; `dict.get(chave, padrao)` resolve o apelido; docstring com bloco `Exemplos` no formato *doctest*.

**Lógica.** Valida o nome, resolve o apelido, despacha para o método e devolve o resultado mônico.

## 8.8 `discretizar_sistema` (linhas 1096–1149)

```
1   φφ→ΠΠΠΠΠ≈≈≈≈
2   def discretizar_sistema(ft_planta, controlador, Ts, metodo="trapezoidal"):
3       """Discretiza planta e controlador separadamente e monta o sistema completo.
4
5       A planta e o controlador são discretizados de forma independente (com o mesmo
6       `Ts` e o mesmo `metodo`) e só então são multiplicados, seguindo
7
8           G_d(z) = discretizar(ft_planta)
9           C_d(z) = discretizar(controlador)
10          T_d(z) = C_d(z) · G_d(z)
11
12      Nota: discretizar a FT contínua já montada (C(s)·G(s)) não é necessariamente
13      equivalente a discretizar as partes separadamente, por isso o produto é feito
14      no domínio discreto.
15
16      Parâmetros
17      ----------
18      ft_planta : TransferFunction ou Expr
19          FT contínua da planta.
20      controlador : Expr ou TransferFunction
21          FT contínua do controlador PI/PD/PID (`Gc`), ou None se não houver controlador.
22      Ts : número real positivo
23          Período de amostragem.
24      metodo : str
25          'euler', 'backward' ou 'trapezoidal'.
26
27      Retorna
28      -------
29      dict
30          Dicionário com as chaves 'planta_discreta', 'controlador_discreto' e
31          'sistema_aberto_discreto' (T_d(z) = C_d(z)·G_d(z), função de transferência
32          em malha aberta).
33      """
34      if controlador is None:
35          raise ValueError(
36              "O controlador não foi calculado. Rode uma das células de PI, PD ou PID "
37              "antes de discretizar o sistema completo."
38          )
39
40      planta_discreta = discretizar(ft_planta, Ts, metodo)
41
42      # Extensão: controlador P é ganho puro (constante), sem variável `s`;
43      # não há o que discretizar, então ele é repassado como está.
44      if sp.sympify(controlador).free_symbols:
45          controlador_discreto = discretizar(controlador, Ts, metodo)
46      else:
47          controlador_discreto = sp.sympify(controlador)
48
49      sistema_discreto = _simplificar(controlador_discreto * planta_discreta)
50
51      return {
```

```
        "planta_discreta": planta_discreta,
        "controlador_discreto": controlador_discreto,
        "sistema_aberto_discreto": sistema_discreto,
    }
```

**Sintaxe.** `if controller is None: raise ...` (falha explícita antes de calcular); `sp.sympify(...).free_symbols` como teste "tem variável contínua?".

**Lógica.** Discretiza planta e controlador *separadamente* e só então multiplica no domínio discreto:

$$G_d(z) = \mathcal{D}[G(s)], \quad C_d(z) = \mathcal{D}[C(s)], \quad T_d(z) = C_d(z)\,G_d(z).$$

Isso não é equivalente a discretizar o produto contínuo já montado, e a docstring registra o porquê. O caso $C(s) = K$ (controlador P) é tratado: sendo ganho puro, não tem o que discretizar e é repassado intacto.

# 10 Extensões

## 9.1 `parametros_padrao` (linhas 1156–1177)

```
φφ→ΠΠΠΠΠ≈≈≈≈
# ---------------------------------------------------------------------------
# 9. Extensões
# ---------------------------------------------------------------------------

def parametros_padrao(Gc, tipo):
    """Extrai (Kp, Ki, Kd) do numerador de 'Gc(s)'.

    P: Kc | PI: Kc·s + Kc·z | PD: Kc·s + Kc·z | PID: Kc·s² + 2Kc·z·s + Kc·z²
    """
    tipo = str(tipo).upper()
    num, _den = sp.fraction(sp.together(_expressao_s(Gc)))
    pol = sp.Poly(sp.expand(num), s)

    c2 = float(pol.coeff_monomial(s ** 2))
    c1 = float(pol.coeff_monomial(s ** 1))
    c0 = float(pol.coeff_monomial(s ** 0))

    if tipo == "P":
        return c0, 0.0, 0.0
    if tipo == "PI":
        return c1, c0, 0.0
    if tipo == "PD":
        return c0, 0.0, c1
    if tipo == "PID":
        return c1, c0, c2
    raise ValueError(f"Tipo de controlador não suportado: {tipo!r}")
```

**Sintaxe.** `coeff_monomial(s**2)` extrai o coeficiente de um monômio específico (retorna 0 se ausente); `str(tipo).upper()`; cadeia de `if` com `return` imediato.

**Lógica.** Lê o numerador de $C(s)$ e devolve $(K_p, K_i, K_d)$:

$$\begin{aligned}
&\text{P:} && K_c \\
&\text{PI:} && K_c s + K_c z \;\Rightarrow\; (K_p, K_i, K_d) = (c_1, c_0, 0) \\
&\text{PD:} && K_c s + K_c z \;\Rightarrow\; (K_p, K_i, K_d) = (c_0, 0, c_1) \\
&\text{PID:} && K_c s^2 + 2K_c z\, s + K_c z^2 \;\Rightarrow\; (c_1, c_0, c_2)
\end{aligned}$$

## 9.2 `polos_malha_fechada` (linhas 1180–1190)

```
φφ→ΠΠΠΠΠ≈≈≈≈
def polos_malha_fechada(ft, controlador):
    """Raízes de '1 + C(s)G(s)H(s) = 0'."""
    expr_ft = _expressao_s(ft)
    expr_gc = _expressao_s(controlador)
    equacao = sp.fraction(sp.together(1 + expr_gc * expr_ft))[0]
    try:
        raizes = sp.nroots(equacao)
```

```
    except (sp.PolynomialError, sp.NoConvergence):
        raizes = sp.solve(equacao, s)
        return [{"valor": complex(r), "exato": sp.sstr(r)} for r in raizes]
    return [{"valor": complex(r), "exato": None} for r in raizes]
```

**Sintaxe.** `sp.nroots` (raízes numéricas) com *fallback* para `sp.solve` (raízes exatas) em `except`; lista de dicionários como retorno.

**Lógica.** Resolve $1 + C(s)G(s)H(s) = 0$ e devolve as raízes. O `fraction(...)[0]` é o numerador dessa soma — que é o polinômio característico. Quando a raiz é exata, vem junto a representação em texto.

## 11 Por que não há `__all__`

O módulo *não* declara `__all__`. A decisão foi verificada em três sentidos:

| Teste | Resultado |
|---|---|
| Alguém usa `from lib import *`? | Não. Só o `app_antigo.py` (backup da v1) fazia isso. |
| Como o `app.py` importa? | Por nome (`from lib import (...)`) e por módulo (`lib.parametros_padrao`). **Nenhuma das duas formas é afetada por `__all__`.** |
| O `__all__` era consistente? | Não. Incluía 4 privados (`_simplificar`, `_expressao_s`, `_variavel_continua`, `_validar_periodo`) e omitia outros 5 (`_avisar`, `_validar_numero`, `_forma_fechada`, `_polinomio_de_coeficientes`, `_discretizar_com`). |

**Lógica.** `__all__` só tem efeito sobre `from lib import *`, e isso não acontece em lugar nenhum do projeto — logo era código morto. Além disso ele não descrevia a API de forma coerente: o critério real de visibilidade já é a convenção do sublinhado inicial, aplicada de forma uniforme em todo o arquivo.

## 12 Mapa de chamadas

| Camada | Funções | Responsabilidade |
|---|---|---|
| Símbolos | `s`, `k`, `z` | variáveis do domínio |
| Utilitários | `_simplificar`, `_expressao_s`, `_variavel_continua`, `coeficientes_ft` | normalização |
| Validação | `_validar_periodo`, `_validar_numero`, `_avisar` | entrada e avisos |
| Ponto $s_i$ | `pole_from_*`, `grandezas_polo` | especificações → polo |
| Planta | `get_ft`, `ganho_planta` | $G(s)H(s)$ |
| LGR | `tabela_lgr`, `angulo_deficit` | passos 11 e 12 |
| Síntese | `_forma_fechada`, `posicoes_zeros`, `posicoes_polos`, `calcular_controlador`, `calcular_controlador_gen` | $C(s)$ |
| Discretização | `_discretizar_com`, `discretizar_*`, `tornar_monico`, `discretizar_sistema` | $C_d(z)$, $G_d(z)$ |
| Extensões | `parametros_padrao`, `polos_malha_fechada` | saídas para o app |

## 13 Desvios em relação ao `pid.ipynb`

| ID | Trecho | Motivo |
|---|---|---|
| D1 | `print` → `_avisar` | o módulo não deve imprimir; o app exibe |
| D2 | `_forma_fechada` | forma fechada isolada e testável |
| D3 | ramo $n_zeros = 0$ | controlador P não existia no notebook |
| D4 | `sigma`, `w_d` opcionais | evita recalcular quando já vêm de $s_i$ |
| D5 | `const` em `discretizar_sistema` | $C(s) = K$ não tem o que discretizar |
| D6 | mensagem de $\varphi$ inatingível | erro claro em vez de `nan` |
| D7 | `polinomio`, `angulo`, `exibir_ft` *não portados* | instrumentação do app: 0 chamadas em 21 execuções |
| D8 | sem `__all__` e sem `calcular_controlador_p` | nada usava `import *`; o atalho do P é o ramo $n_zeros = 0$ |